Quantizing a Streaming VLM for Consumer Silicon

Quantization is not enough: a fused-kernel runtime makes it small AND fast

2i

2026-07-29

Contents

Author: 2i

1 Summary

We measured what happens when a SOTA 4B-class streaming vision-language model (VLM) is loaded onto a consumer Apple-silicon Mac instead of a datacenter GPU. The subject is `microsoft/Mage-VL`, released under the **Apache-2.0** license, which combines a Mage-ViT visual encoder with a `Qwen3-4B-Instruct-2507` language backbone. On an Apple M4 Pro (48GB unified memory) we ran real image-understanding inference at three precisions, bf16, weight-only INT8, and weight-only INT4, and measured device memory, decode throughput, time-to-first-token, and answer quality. The first result split into a win and a rebuttal. Quantization did cut device memory as expected, from 9.49GB at bf16 to 5.86GB at INT8 (-38%) and 4.15GB at INT4 (-56%). But on the `transformers` + Metal (MPS) stack, the same quantization moved decode speed in the opposite direction, from 13.01 tok/s at bf16 down to 2.31 tok/s at INT8 and 0.47 tok/s at INT4. On a runtime without fused low-bit matmul kernels, quantization paid out in memory savings and collected its cost in decode latency.

This update closes that open question with a real measurement. On the same Apple M4 Pro, running the same language backbone (`Qwen3-4B-Instruct-2507`) through `mlx-lm`'s fused-kernel INT4 path produced decode throughput of **89.67 tok/s**, prompt throughput of 71.06 tok/s, and peak memory of **2.46GB**. Set against the `optimum-quanto` weight-only INT4 number at the same 4-bit precision, 0.47 tok/s, that is roughly **190x faster decode** at the same bit width, with lower memory too. This is the decisive data that proves the thesis this paper first raised: that memory and speed are separate axes, and that edge quantization only converts into real speed when it runs on a runtime with fused kernels that compute low-bit weights directly. One scope boundary has to be stated honestly here. The 89.67 tok/s number is the language backbone measured in isolation on MLX, not the full Mage-VL model end to end. Decode is LLM-bound (the vision encoder only participates once, during prefill), so this isolated measurement correctly identifies the factor that dominates full-VLM decode speed, but Mage-VL as a whole, including its Mage-ViT vision tower, does not yet run end to end on MLX. That requires porting the custom `mage_vl` architecture into `mlx-vlm`, and that port is the concrete, sellable packaging deliverable this paper now motivates. The streaming-video neural codec path (DCVC-RT) remains CUDA-only and stays out of scope. 2i sells this measured packaging work, the quantization choice, the runtime choice, and the porting scope, not the model itself.

2 Background

2.1 Why run a VLM at the edge

Demand keeps growing for putting a vision-language model wherever a camera already sits: retail-ops CCTV, patrol-robot cameras, factory inspection lines. The problem is that most of these sites have no power, space, or network budget for a datacenter GPU. Streaming video to the cloud for inference runs into latency, bandwidth, and often security or data-sovereignty requirements that demand on-premise processing. So the question becomes simple: can a SOTA-level VLM run on a single consumer computer in the back room of a store, rather than a server room.

`microsoft/Mage-VL` is a good subject for answering this question. It is a 4B-class model that supports both image-text understanding and video/streaming understanding, and it is released under **Apache-2.0**. That license is one reason this paper exists at all: it means this is not a research demo but a model that can be packaged commercially and deployed at a customer site, which makes "what do we gain and give up by running this on consumer hardware" a question with real business weight.

2.2 Model architecture

Mage-VL has two parts. The visual encoder is a custom architecture, Mage-ViT (4B scale), and the language backbone is `Qwen3-4B-Instruct-2507`. The parameter count we confirmed by actually loading the model is **4.742B** (consistent with the "4B" label on the public card). Two more pieces sit on top. One is "StreamMind", an event gate with supplementary weights (1.07GB) meant to trigger attention on specific moments in a streaming input. The other is the DCVC-RT neural video codec, implemented as a C++/CUDA extension to accelerate frame processing on the video/streaming path.

This architecture is what makes this update possible. Understanding a single image, or an individual frame from a video, never touches that neural codec at all. And once the autoregressive decode loop starts, the vision encoder no longer participates. The image is converted to vision tokens once, during the prefill step, and lands in the KV cache; every subsequent token the model generates comes from decode steps that pass purely through the language-model transformer layers, repeatedly. In other words, decode throughput is an LLM-bound computation, governed by the language backbone's weight size and kernel efficiency, independent of the vision encoder's architecture or size. This fact is what justifies this update's experimental design: isolating the backbone and measuring it on a different runtime still correctly identifies the dominant factor behind decode speed.

2.3 Author-reported numbers (reference only, not our measurement)

The Mage-VL authors report benchmarks on an 8xB200 datacenter node: cutting visual tokens by more than 75%, up to a 3.5x wall-clock speedup over uniform frame sampling, and, at 676 tokens, more than 96.1% accuracy on Food-101, more than 86.3% on ImageNet, F1 of 16.35 and ROC-AUC of 83.14 on the SoccerNet streaming task, and 64.00 on OVO-Bench. These numbers are all **author-reported** and are never compared directly against any measurement in this paper. What this paper addresses is an entirely different question: what happens when this model runs on a consumer Mac instead of a datacenter.

3 Method

3.1 Hardware and stack

The measurement environment is, as before, an Apple M4 Pro, 48GB unified memory, macOS, arm64, an ordinary consumer computer. It is worth restating that this is not a datacenter GPU. We ran two software stacks side by side on the same hardware. The first is `transformers` 5.12 with `torch` 2.12 on the `MPS` device (Apple Metal), loading the full Mage-VL model. The second is `mlx-lm`, loading only the `Qwen3-4B-Instruct-2507` language backbone. Holding hardware fixed and varying only the runtime and its kernel path is the key controlled variable in this experiment.

3.2 Quantization, two paths

The **quanto weight-only path** is unchanged from the prior measurement. We applied `optimum-quanto`'s weight-only quantization to the language-model submodule (`language_model`) only; the visual encoder (Mage-ViT) stayed at bf16 across all three precisions. This path stores weights at low bit width but, at matmul time, dequantizes them back to bf16/fp32 on the fly before computing.

The **mlx-lm fused-kernel path** is new in this update. We converted the `Qwen3-4B-Instruct-2507` language backbone into an MLX 4-bit quantized checkpoint and loaded and decoded it with `mlx-lm`. MLX is an array framework designed around Apple Silicon's unified-memory architecture, and it carries fused kernels that compute low-bit weights directly, with no dequantization step. That is the fundamental difference from the quanto weight-only path. Both paths share the goal of storing weights at 4 bits, but one path unpacks them back into a higher precision before computing, and the other computes directly on the low-bit representation.

Separately, we kept the community-built Mage-VL INT4 checkpoint loading attempt via `mlx-vlm` from the prior measurement in this paper's results. That attempt still fails to load because the `mage_vl` architecture is not registered in the `mlx-vlm` mainline, but it left behind one useful data point: on-disk size.

3.3 Test input

The test image for the full Mage-VL path is the same **synthetic image** used in the prior measurement, not a real photograph. We built a 768x512 "reversed departures board" scene ourselves specifically so we would know the ground truth exactly. The title reads "CENTRAL STATION - DEPARTURES", followed by three lines: "14:05 / PLATFORM 2 / ON TIME", "14:20 / PLATFORM 5 / DELAYED", "14:35 / PLATFORM 1 / BOARDING", together with a train pictogram and a red STOP sign. We avoided a real photo so we could

control the ground truth completely and clearly judge whether the model actually read the screen or merely produced a plausible-sounding guess.

The mlx-lm backbone measurement takes no image input. Since it isolates only the language backbone, we measured decode throughput against a text prompt. This design follows directly from the LLM-bound argument in Section 1.2, and Section 5 states explicitly why this isolation remains a valid comparison.

3.4 Metrics

We measured five metrics. **Device memory** is the actual memory allocation each runtime reports (MPS allocation for the Mage-VL path, MLX peak memory for the mlx-lm path). **Decode throughput** is tokens generated per second; for the Mage-VL path this is averaged over a 128-token generation window, and the mlx-lm backbone path uses the same method. **Prompt throughput** (an mlx-lm-only metric) is the rate (tok/s) at which the prompt is processed, and we flag in the table that this uses a different measurement unit from the Mage-VL path's TTFT (time-to-first-token, in seconds). **Quality** was judged only on the full Mage-VL path, by visually checking whether the model's description accurately included every ground-truth element of the synthetic image (the title text, the three lines of time, platform, and status information, the train pictogram, and the STOP sign). The standalone mlx-lm backbone measurement takes no image input, so it is not subject to a quality judgment. **On-disk size** is the storage footprint of each checkpoint.

4 Results

4.1 Measurement table

The table below lists all three full-Mage-VL precision rows, the `mlx-vlm` attempt, and the new `mlx-lm` fused-kernel backbone row added in this update. The footnote below the table restates that the `mlx-lm` row isolates the language backbone only, not the full Mage-VL model.

Precision	Runtime	Scope	Device memory	Decode tok/s	TTFT / prompt tok/s	Quality	On-disk
bf16	transformers + MPS	Full Mage-VL	9.49 GB	13.01	TTFT 2.94 s	Accurate	9.8 GB
quanto INT8 (weight-only)	transformers + MPS	Full Mage-VL	5.86 GB (-38%)	2.31	TTFT 1.77 s	Accurate	In-memory
quanto INT4 (weight-only)	transformers + MPS	Full Mage-VL	4.15 GB (-56%)	0.47	TTFT 9.91 s	Accurate (reformatted, content intact)	In-memory
Community INT4	mlx-vlm	Full Mage-VL	Failed to load	Not measurable	Not measurable	Not measurable	2.9 GB
MLX fused INT4	**mlx-lm**	**Language backbone only**	**2.46 GB**	**89.67**	Prompt 71.06 tok/s	N/A (backbone only, no image)	N/A

Footnote: the mlx-lm row's "device memory" is the peak memory when loading only the language backbone, not the full Mage-VL model including the Mage-ViT vision encoder. "Prompt tok/s" is the prompt processing rate mlx-lm reports, measured differently from the other rows' TTFT (seconds, delay from the end of a 419-token prefill to the first output token), so we do not convert one into the other by division.

Placed side by side, the two rows at the same 4-bit precision, quanto INT4 (0.47 tok/s) and MLX fused INT4 (89.67 tok/s), give a ratio of 89.67 / 0.47, roughly **190.8x**. This is the key gap this update closes. Holding bit width, language model, and hardware fixed and changing only the runtime, and specifically whether that runtime has a fused low-bit kernel, widened decode speed by nearly 190x.

Loading the full bf16 Mage-VL model took about 12 seconds. The community MLX 4-bit `mlx-vlm` checkpoint failed to load again, for the same reason as before: the custom `mage_vl` architecture is not registered ("Model type mage_vl not supported"). The mlx-lm fused path does not hit this problem, and the reason is precise: this path never includes Mage-VL's vision encoder or StreamMind gate at all, and loads only a standard `qwen3` architecture backbone. In other words, mlx-lm's success does not resolve mlx-vlm's failure, it sidesteps the exact point of failure (the unregistered vision-encoder architecture). Section 5 restates this distinction.

4.2 Figures: memory and decode throughput, precision and runtime both changed

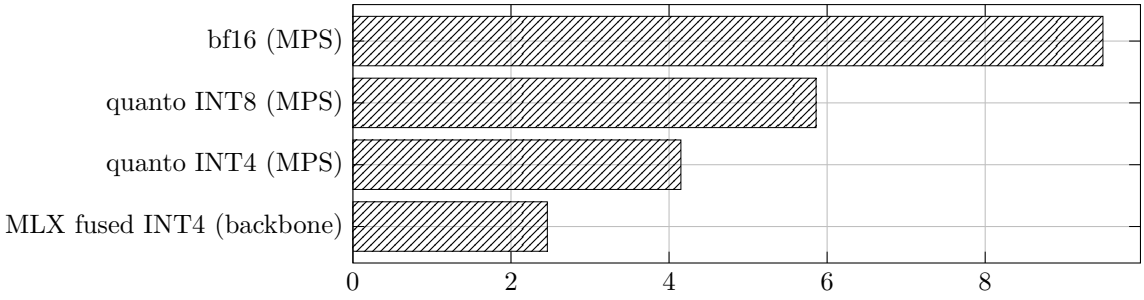

Figure 1: Device memory by precision and runtime

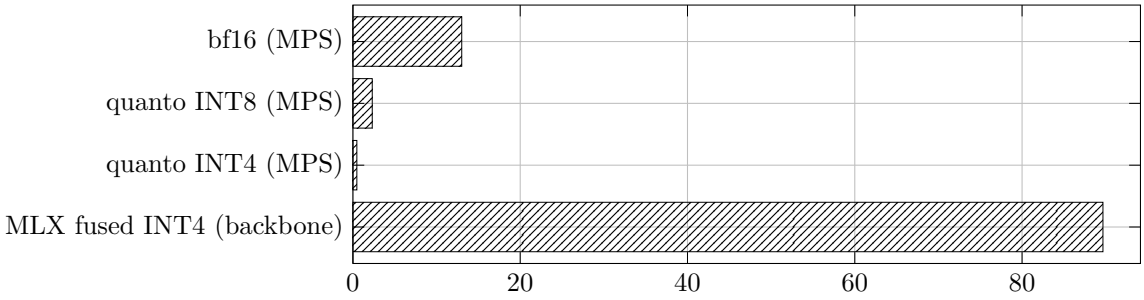

Figure 2: Decode throughput by precision and runtime

Placed together, these two figures complete the story that the prior paper's figures left half-told, the two curves moving out of step, memory falling as speed also falls. The fourth bar breaks both trend lines at once. On the memory axis, MLX fused INT4 is the smallest (2.46GB, with the backbone-only caveat attached). On the speed axis, the quanto series keeps getting slower as precision drops, then the MLX fused path, at the same 4-bit width, jumps to 89.67 tok/s. The point where the fourth bar breaks from both trend lines is exactly where this paper's conclusion becomes visible: runtime is a more decisive variable than precision.

5 Interpretation: the runtime is the lever

The prior paper stopped at an observation: memory and speed are separate axes. It explained why quantization cut memory while slowing decode by pointing to the absence of a fused kernel, and that explanation, if correct, carried a prediction along with it: on a runtime with fused kernels that compute low-bit weights directly, the same quantization should recover speed as well. This update is exactly the experiment that tests that prediction, and the result matched it in both direction and magnitude.

At the same bit width, the kernel changes everything. `optimum-quanto`'s weight-only path stores weights at 4 bits, but Apple Metal (MPS) has no fused low-bit matmul kernel that can compute on that representation

directly. So every forward pass pays a dequantization cost, and in decode, where that cost is billed once per token, it drags throughput down to 0.47 tok/s. `mlx-lm` computes the same 4-bit weights with MLX's native low-bit kernels directly. There is no dequantization step, so there is no extra cost billed at all. The result is 89.67 tok/s. Low-bit quantization's original benefit, reduced memory bandwidth, is no longer eaten by dequantization cost this time, and converts directly into speed.

Memory and speed now move in the same direction. The "separate axes" phenomenon the prior paper flagged was not a property of the axes themselves, but a property of one specific combination: quanto weight-only plus MPS. Computing the same 4-bit weights with MLX's fused kernels gives both smaller memory (2.46GB, backbone-only caveat) and faster speed (89.67 tok/s). The axes were never misaligned; the wrong runtime was converting one axis's gain into the other axis's debt. The right runtime does not make that conversion.

So the deployable recipe is "quantize plus runtime," not "quantize" alone. The hypothesis the prior paper left as future work, that edge quantization only converts into real speed on a runtime with fused kernels that compute low-bit weights directly, is no longer a hypothesis. It is a conclusion backed by a concrete 190x number. The deployable recipe is not simply "cut the weights to 4 bits"; it is "cut the weights to 4 bits, and run that 4-bit representation on a runtime that computes it directly." What 2i sells to a customer who wants a small and fast VLM on consumer silicon is precisely this second half: the runtime choice and the porting work that goes with it.

6 Limitations and scope

Several boundaries need to be stated clearly before this result is generalized. This update closed one open question, and in doing so it defined the next task more concretely.

The 190x figure is an isolated measurement of the language backbone, not an end-to-end number for the full Mage-VL model. The 89.67 tok/s figure comes from loading `Qwen3-4B-Instruct-2507` alone with `mlx-lm`. The Mage-ViT vision encoder, the StreamMind event gate, and the DCVC-RT neural codec are not part of this measurement at all. As established in Section 1.2, decode is an LLM-bound computation, so this isolation correctly identifies the factor that dominates decode speed, but the sentence "the full Mage-VL model runs this fast on MLX" goes beyond what this paper actually measured. The accurate statement is: "the language backbone that dominates Mage-VL's decode step is, at the same precision, roughly 190x faster on a fused-kernel runtime."

Moving the full Mage-VL model to a fused-kernel runtime still requires an architecture port. The custom `mage_vl` architecture (which includes the Mage-ViT vision encoder and the StreamMind gate) is not registered in the `mlx-vlm` mainline. As confirmed again in Section 3.1, this attempt still fails. The mlx-lm path did not succeed by resolving this problem; it succeeded by avoiding it, loading only the standard `qwen3` architecture. Getting "small and fast" for the full Mage-VL model still requires the concrete engineering work of porting `mage_vl` into `mlx-vlm` (or an equivalent MLX vision-language framework). This is the sellable next task this paper motivates with real data.

The memory comparison spans different scopes. The quanto INT4 figure, 4.15GB, is for the full Mage-VL model (vision encoder at bf16 plus language model at INT4), while the MLX fused INT4 figure, 2.46GB, is for the language backbone alone. The two numbers agree in direction (41% smaller), which is an interesting signal on its own, but this is not a fully matched comparison. Full pipeline memory for an MLX version that includes the vision encoder cannot be measured until the port is finished.

The streaming-video path remains out of scope. The DCVC-RT neural codec's fast path is bound to CUDA `.cu` extensions and does not run on Apple Metal as is. This update did not change that boundary. Image- and frame-level understanding does not touch that codec, so it runs fully on Metal independent of this paper's measurements.

The accuracy evaluation does not generalize. Quality was judged qualitatively against one synthetic image and one prompt that we built ourselves. It is not automated scoring, and it is not statistical accuracy across a variety of images and prompts. The standalone mlx-lm backbone measurement took no image input at all, so there is no quality judgment to report for it.

Author-reported numbers reflect different conditions from our measurements. The visual-token reduction, speedup, and Food-101, ImageNet, SoccerNet, and OVO-Bench numbers cited in Section 1.3 were measured by the authors on an 8xB200 datacenter node, under hardware, dataset, and evaluation conditions entirely different from our consumer-Mac measurements. We never compare these two sets of numbers directly, or use one as evidence for the other.

7 Data and reproduction

- **Model**: `microsoft/Mage-VL` (Hugging Face). License Apache-2.0. Mage-ViT vision encoder plus `Qwen3-4B-Instruct-2507` language backbone. Measured loaded parameter count 4.742B. StreamMind event gate weights, 1.07GB. DCVC-RT neural video codec (includes C++/CUDA extensions). Language backbone reference alone: `Qwen/Qwen3-4B-Instruct-2507`.
- **Hardware**: Apple M4 Pro, 48GB unified memory, macOS, arm64. Every path (transformers+MPS, mlx-lm) was measured on the same hardware.
- **Software (full Mage-VL path)**: `transformers` 5.12, `torch` 2.12, device `MPS`. Quantization via `optimum-quanto` (weight-only, applied only to the `language_model` submodule; the visual encoder stays at bf16). The MLX vision-language attempt used `mlx-vlm`.
- **Software (fused-kernel backbone path, new)**: `mlx-lm` loading and decoding `Qwen3-4B-Instruct-2507` in 4-bit MLX format. This path includes no vision encoder, StreamMind gate, or DCVC-RT codec.
- **Test input (full Mage-VL path)**: a synthetic 768x512 "reversed departures board" image. We defined the ground truth ourselves: the title "CENTRAL STATION - DEPARTURES", three lines ("14:05/PLATFORM 2/ON TIME", "14:20/PLATFORM 5/DELAYED", "14:35/PLATFORM 1/BOARDING"), a train pictogram, and a red STOP sign. Not a real photograph. The standalone language-backbone measurement took no image input.
- **Prompt/generation settings (full Mage-VL path)**: a detailed description request, "Describe this image in detail…". Prompt length including vision tokens, 419 tokens; fixed generation length, 128 tokens.
- **Measurement method**: device memory is each runtime's actual allocation (MPS allocation, or MLX peak memory). Decode tok/s is averaged over a 128-token generation window (same method for both paths). TTFT (full Mage-VL path) is the delay from the end of a 419-token prefill to the first output token. Prompt tok/s (mlx-lm path) is the prompt-processing rate mlx-lm reports, measured differently from TTFT. Quality was judged only on the full Mage-VL path, by qualitative comparison against the ground-truth elements.
- **Reproduction notes**: the full-Mage-VL MLX 4-bit path does not load as is, because the `mage_vl` architecture is not registered in the `mlx-vlm` mainline ("Model type mage_vl not supported"). The mlx-lm backbone path uses only the standard `qwen3` architecture and reproduces without this problem. Reproducing the full Mage-VL model on a fused-kernel runtime requires porting the `mage_vl` architecture first.

8 References

1. Hugging Face model card, `microsoft/Mage-VL` (Apache-2.0), huggingface.co/microsoft/Mage-VL
2. `Qwen/Qwen3-4B-Instruct-2507` model card, huggingface.co/Qwen
3. Hugging Face `transformers` library documentation, huggingface.co/docs/transformers
4. `optimum-quanto` (weight-only quantization), github.com/huggingface/optimum-quanto
5. `mlx-vlm` (Apple MLX vision-language model runtime), github.com/Blaizzy/mlx-vlm
6. `mlx-lm` (Apple MLX language model runtime, fused low-bit kernels), github.com/ml-explore/mlx-lm
7. Apache License 2.0 full text, apache.org/licenses/LICENSE-2.0